

DTE Platform - V1 Implementation Roadmap

As your Technical Delivery Lead and Principal Engineer, my mandate is to transition this platform from approved architecture into physical reality. This roadmap is the definitive execution plan.

With a small founding team, we cannot afford horizontal "layer-by-layer" development. We will build vertically, proving the highest-risk integrations first, and deferring edge cases until the core engine can successfully transmit a document to the Ministry of Finance (MH).

Here is the construction sequence for V1.

1. Guiding Principles

- **Vertical Slices over Horizontal Layers:** We do not build "all the database tables" and then "all the APIs." We build a single document type (FE) from Draft to Accepted, touching every architectural layer.
- **Secure the Perimeter Early:** The MH API and the Cryptographic Signing Vault are our two biggest external dependencies. They must be integrated and proven by Sprint 3.
- **Fake the UI, Prove the API:** The UI team will build against the Swagger/OpenAPI spec using mock servers. The backend team will not wait for the UI to validate business logic.
- **Audit is Immediate, Not an Afterthought:** The tamper-evident audit trail and PostgreSQL append-only constraints are built alongside the very first command. If we retrofit security, we fail compliance.

2. Dependency Graph

To prevent blocked developers, work must flow according to this strict dependency chain:

1. **Level 0 (Foundation):** Solution Structure, NetArchTest rules, Docker-Compose (PostgreSQL + Hangfire).
2. **Level 1 (Core Domain):** FiscalDocument Aggregate, State Machine, Value Objects.
3. **Level 2 (Persistence & Application):** EF Core Config (JSONB mappings), MediatR Commands/Queries.
4. **Level 3 (Integration):** Signing Service implementation, MH API Client.
5. **Level 4 (Background):** Hangfire queues for Transmission.
6. **Level 5 (Exceptional Domains):** Invalidation, Contingency.
7. **Level 6 (Presentation):** UI Wiring, PDF Representation Generation.

3. Build Order & 4. Vertical Slice Strategy

We will attack the implementation in distinct vertical slices, deliberately ignoring complex features until the primary objective is achieved.

- **Slice 1: The "Walking Skeleton" (FE Draft $\rightarrow$ Validated)**
 - *Goal:* Prove the CQRS architecture, Domain validation rules, and PostgreSQL JSONB snapshotting for a basic *Factura Electrónica* (01).
- **Slice 2: The Cryptographic Freeze (Validated $\rightarrow$ Signed)**

- *Goal:* Prove the signing integration and the creation of the immutable EvidenceBundle using PostgreSQL BYTEA.
- **Slice 3: The MH Handshake (Signed $\rightarrow$ Accepted)**
 - *Goal:* Prove Hangfire background processing, MH API authentication, and recording the Acceptance Stamp.
- **Slice 4: The Strict Sibling (CCF)**
 - *Goal:* Introduce the stricter validation rules for *Comprobante de Crédito Fiscal* (03) utilizing the same state machine.
- **Slice 5: The Exceptions (Invalidation & Contingency)**
 - *Goal:* Implement the regulatory edge cases now that the happy path is ironclad.

5. Milestone Breakdown & 6. Sprint-by-Sprint Roadmap

Assumes a 2-week sprint cycle for a small founding team.

Milestone 1: Core Engine & Identity

- **Sprint 1: The Foundation & Draft Workflow**
 - Setup .sln, project boundaries, and Testcontainers.
 - Implement FiscalDocument aggregate and CreateDraftCommand.
 - Implement ValidateDocumentCommand (assigning ControlNumber safely).
 - *Demo:* API creates a Draft and transitions it to Validated, returning a JSON snapshot.
- **Sprint 2: Signing & Evidence Persistence**
 - Implement Database triggers blocking UPDATE/DELETE on evidence tables.
 - Implement SignDocumentCommand.
 - Mock the external Vault; generate the canonical JSON payload and store it in BYTEA.
 - *Demo:* A document is cryptographically frozen; the DB physically rejects tampering attempts.

Milestone 2: The Ministry Handshake

- **Sprint 3: Transmission & Hangfire**
 - Deploy Hangfire via PostgreSQL.
 - Implement TransmitDocumentCommand (places document in SUBMITTED).
 - Build the MH API HTTP Client (Authentication + Send).
 - *Demo:* A Hangfire worker picks up a Signed document, pings a test MH endpoint, and records a TransmissionRecord.
- **Sprint 4: Audit Trail & Catalogs**
 - Implement the Hash-Chained AuditEntry pipeline behavior in MediatR.
 - Build CRUD for Counterparty and Item catalogs.
 - *Demo:* Every action now produces a verifiable cryptographic ledger entry.

Milestone 3: Compliance & Exceptions

- **Sprint 5: Security & The CCF**

- Implement JWT Authentication and Owner/Biller RBAC.
- Introduce Document Type 03 (CCF) into the Registry; enforce strict taxpayer validation.
- *Demo*: Billers are blocked from config; CCFs are rejected if missing NIT/NRC.
- **Sprint 6: Plain Invalidation**
 - Implement InvalidationEvent aggregate.
 - Implement RequestInvalidation and TransmitInvalidation commands.
 - *Demo*: An Accepted document is successfully transitioned to INVALIDATED without destroying its original evidence.

Milestone 4: Offline Survival & Polish

- **Sprint 7: Contingency Episode**
 - Implement OpenEpisode command.
 - Route newly signed documents to CONTINGENCY_HELD.
 - Implement TransmitNotification (Reconciliation batching).
 - *Demo*: System survives network outage and replays documents in legal order.
- **Sprint 8: UI Integration & Human-Readable PDF**
 - Implement the deterministic PDF generation service (stored in Evidence).
 - Connect the Frontend UI to the hardened API.
 - *Demo*: Full UI-driven issuance of a real DTE.

7. Demo Milestones

To maintain stakeholder confidence, demos must showcase *provable system states*, not UI wireframes.

- **Sprint 2 Demo**: "Here is the database physically rejecting an SQL UPDATE to a signed document."
- **Sprint 3 Demo**: "Here is the exact network payload we sent to the MH, and their exact JSON response, permanently locked in our Transmission Record."
- **Sprint 6 Demo**: "Here is the cryptographic hash chain proving exactly who invalidated this document."

8. Technical Risk Reduction Strategy

- **Testcontainers from Day 1**: Do not use Entity Framework In-Memory providers. They do not enforce JSONB constraints or Triggers. All integration tests must run against a real PostgreSQL container.
- **Hangfire Idempotency**: The highest technical risk is sending the same document to the MH twice. We will write explicit integration tests simulating network timeouts to ensure Hangfire locks prevent duplicate transmissions.
- **Concurrency Testing**: Write tests that hammer the ValidateDocumentCommand with 50 concurrent requests for the same POS to ensure the ControlNumberSeries sequence generator never produces a gap or duplicate.

9. Regulatory Risk Reduction Strategy

- **Schema First**: Download the official MH JSON Schemas. Run our generated canonical payloads against these official schemas in our Unit Tests *before* ever attempting an HTTP

call to the MH.

- **Early Sandbox Access:** By Sprint 3, the platform must be communicating with the MH Testing/Sandbox environment. We will not wait for UI completion to validate our payload shapes against the regulator.

10. Definition of Done (DoD) per Milestone

A feature is not "Done" when the code is written. It is Done when:

1. **Architectural Tests Pass:** NetArchTest confirms no boundary violations.
2. **Audit Verified:** The action successfully appended a verifiable hash to the Audit Trail.
3. **Evidence Locked:** The output (if applicable) is safely stored in a BYTEA column.
4. **Integration Tests Pass:** Tested against a live PostgreSQL Testcontainer.
5. **Idempotency Confirmed:** The command can be executed twice without corrupting state.

11. First Customer Deployment Path

We optimize for a "White-Glove" self-hosted deployment for Customer #1:

- **Infrastructure:** A single Ubuntu Linux VPS.
- **Deployment:** docker-compose up -d running the API, Hangfire worker, and PostgreSQL in distinct containers.
- **Configuration:** IssuerConfiguration and Certificate references are seeded manually via a secure API endpoint or DB script by the Owner. No complex onboarding wizard will be built for V1.
- **Proxy:** NGINX acting as the reverse proxy handling TLS termination.

12. What MUST NOT be Built Early

To ensure we hit our release dates, strictly forbid the team from building the following until Sprint 7/8 (or V1.5):

- **PDF Generation:** Do not waste weeks perfectly aligning pixels on a PDF invoice in Sprint 2. Use raw JSON for validation. Add the PDF generation right before UI integration.
- **Complex UI Dashboards:** Stick to flat data tables for the UI.
- **Email Delivery:** V1 scope does not require emailing the receptor. Do not build SMTP integrations.
- **Automated Certificate Rotation:** Manual updates to the configuration vault are acceptable for V1.

13. Recommended Development Sequence (Layer by Layer)

When a developer picks up a feature (e.g., CreateDraft), they must code in this exact order:

1. **Domain:** Write the State Machine rules, Aggregate methods, and Unit Tests.
2. **Application:** Write the MediatR Command, CommandHandler, and Validation logic.
3. **Infrastructure:** Write the EF Core configuration (IEntityTypeConfiguration) and mapping for the Aggregate.
4. **Web API:** Add the Controller endpoint, mapping the HTTP POST to the MediatR

Command.

Never start at the Controller and work downward.

14. Path to First Production DTE Accepted by MH

This is the golden thread. To achieve the first legally valid production DTE, we will execute this exact sequence:

1. **Provision:** Customer #1 obtains their MH Production Credentials and .pfx certificate.
2. **Configure:** The Owner logs in and saves the IssuerConfiguration, Establishment (0001), and Point of Sale (001) data.
3. **Draft:** Biller uses the UI to create a Draft FE (Type 01).
4. **Validate:** API validates totals, locks the master data snapshots, and assigns the first legal Control Number (DTE-01-0001001-0000000000000001).
5. **Sign:** API requests a signature from the vault. The Canonical JSON is signed, hashed, and committed to the EvidenceBundle as a PostgreSQL blob.
6. **Transmit:** Hangfire picks up the SUBMITTED document, authenticates with the MH API, and pushes the payload.
7. **Acceptance:** The MH API responds with PROCESADO and a Sello de Recepción.
8. **Lock & Audit:** The Acceptance Stamp is appended to the Evidence Bundle. The final Audit Entry is hash-chained. The document is permanently, legally valid.